Refactor and Optimization Report

MEASURED CLEANUP - FOR TEAM REVIEW

Owner: Khalil Dridi

Audit date: 2026-10-31

Status: Two focused lint cleanups implemented and tested; before/after evidence recorded. Ready for team review. Publication, PR/CI and final human acceptance remain pending.

Scope: Existing repository implementation only. No new features, model changes, frozen-evidence changes, or protected-dataset access.

Assignment and submission context

The midterm progress submission (due Monday, 2 November 2026 at 8:00 AM) asks the team to provide a runnable MVP, a recorded demonstration, repository access, and all assignments completed so far. One team member submits the repository URL, demo link, and all team members' names; assignment files must be attached or committed to the repository.

This report is Part 6, Khalil Dridi's lead contribution. The team division assigns Iheb to provide API/UI metrics, Donia to provide ML metrics, and Malak to edit the final document. Part 3 (the UML package) was allocated jointly to Khalil and Malak; Khalil reports that Malak completed that work. This clarifies Khalil's remaining assigned deliverable as Part 6; it does not replace the team's other submission requirements.

Baseline

The latest GitHub main tip observed for this audit is 98fd8d7e17f81ff1f9687f6ea3a83804a50fa628 (merge of PR #71). Entry 107 in the project log records the following check results:

```
Command: python -m pytest -q
Output: 794 passed, 9 skipped in 21.99s

Recorded check: tests/governance/test_deepvoice_access.py (the exact invocation was not preserved in
Entry 107)
Output: 3 passed in 0.59s

Command: git diff --check
Output: passed

Token scan (exact invocation not preserved in Entry 107)
Output: no token found
```

The results above are historical repository records. The fresh baseline below was run by Khalil on `docs/stage11-midterm-refactor-khalil`, whose application source matched main `98fd8d7e17f81ff1f9687f6ea3a83804a50fa628` at audit time. Documentation changes existed during the measurements; the evidence summary records their provenance. Khalil fetched and created the branch in PowerShell. The initial report was committed as `7c0e2437b63cd8eb5691184625bbc5813afdeec1`, and the measured baseline as `3f57ec3ba4c7ba677e82de5dae7891d7dc96c98a`. At this pre-push review, publication and PR review have not yet occurred.

Khalil subsequently ran fresh checks in his own PowerShell after installing `requirements.txt`. The pasted output and screenshot show:

```
Command: .\.venv\Scripts\python.exe -m pip check
Output: No broken requirements found.

Command: .\.venv\Scripts\python.exe -m pytest -q -ra
Output: 794 passed, 9 skipped in 51.00s
Skipped: 8 because torch is not installed; 1 because wandb is not installed.

Command: .\.venv\Scripts\python.exe -m pytest -q tests/governance/test_deepvoice_access.py
Output: 3 passed in 0.22s
```

The first full run above lacked the CNN dependencies. Khalil then installed `torch==2.11.0` from the official CPU index (installed version `2.11.0+cpu`), followed by `requirements-cnn.txt` (`safetensors==0.8.0`, `wandb==0.31.0`). `pip check` again returned `No broken requirements found.` The complete baseline then passed:

```
$env:REQUIRE_TORCH_TESTS = "1"
.\.venv\Scripts\python.exe -m pytest -q -ra

937 passed in 89.85s (0:01:29)
```

The final summary shows no failures, errors or skips. These are user-executed results supplied by pasted output and screenshots, not assistant-executed runs. Environment: local Windows, CPU PyTorch; `python --version` printed Python 3.13.16 before the coverage-instrumented run. The earlier eight torch skips were module-level skip records, not eight individual test functions; installing the missing dependencies enabled additional collected tests. The initial missing-FastAPI collection error and the 794-pass run are superseded by the complete 937-pass baseline. Installing declared dependencies is environment repair, not a source-code refactor or speed improvement. The difference between the two suite durations cannot support a speed comparison because the executed tests differ.

Full test suite

Before result: Fresh complete user run: 937 passed in 89.85s (0:01:29), with REQUIRE_TORCH_TESTS=1 and no skips shown

Governance test

Before result: Baseline: 3 passed in 0.22s; final documentation check: 3 passed in 0.18s (user-run)

Coverage

Before result: Statement 88.92% (3,748/4,215); branch 79.80% (1,047/1,312); combined 86.76%. Coverage.py 7.17.0, api/ and src/ only

Lint/format

Before result: Ruff 0.17.3: 2 findings under explicitly selected E4/E7/E9/F rules; no source fix performed. Git diff check passed. No formatter assessment

Complexity

Before result: Radon 6.0.1: 342 function/method blocks, mean 4.6082, maximum 37 (rank E); aggregation defined below

API latency/resource use

Before result: Not measured before; status endpoint is mock, and the API design leaves the latency target open

No DEEP-VOICE or FoR test audio was accessed for this audit.

Final documentation check supplied by Khalil after the report and documentation records were added:

```
.\.venv\Scripts\python.exe -m pytest -q tests/governance/test_deepvoice_access.py

3 passed in 0.18s
```

The earlier 0.22 s governance result is retained above as a separate baseline run. The final check covers the documentation working tree. `git diff --check` passed; all relative links in the report resolve in the project checkout, no literal-token patterns were found in the four changed documents, and existing project-log entries 1-107 were confirmed unchanged.

Measured software baseline

These are user-executed measurements from the current Python 3.13.16 project environment, before any code refactor. Coverage 7.17.0, Radon 6.0.1 and Ruff 0.17.3 versions are captured in the raw evidence. At the before measurement, the `api/` and `src/` source code was identical to base commit `98fd8d7`; only documentation had changed at that point. Summary and provenance (audit_evidence/midterm_refactor/before/summary.json) records the exact source/base distinction, tool scope and raw file hashes.

```
$env:REQUIRE_TORCH_TESTS = "1"
$env:COVERAGE_FILE = Join-Path $part6Out ".coverage"
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m coverage run --branch --source=api,src -m pytest -q -ra
.\.venv\Scripts\python.exe -m coverage report --precision=2
.\.venv\Scripts\python.exe -m coverage json -o (Join-Path $part6Out "coverage.json")

Python 3.13.16
937 passed in 73.21s (0:01:13)
TOTAL  4215  467  1312  191  86.76%
```

The TOTAL columns are statements, missed statements, branches, partially covered branches, and combined coverage. The coverage JSON (audit_evidence/midterm_refactor/before/coverage.json) separately records statement coverage **88.9205%**, branch coverage **79.8018%**, and combined coverage **86.7559%**. Each exceeds the course's 60% target for this stated source scope; this does not mean every file has 60% coverage. Notable gaps remain: `src/preprocessing/benchmark.py` has combined coverage 19.12%, and `src/preprocessing/validation.py` 47.12%. A passing coverage target does not prove correctness or authorize real-data execution to increase coverage.

```
$env:RADONFILESENCODING = "UTF-8"
.\.venv\Scripts\python.exe -m radon cc -j api src | Set-Content -Encoding utf8 (Join-Path $part6Out
"complexity.json")
.\.venv\Scripts\python.exe -m ruff check --isolated --no-cache --target-version py313 --select
E4,E7,E9,F --output-format json api src | Set-Content -Encoding utf8 (Join-Path $part6Out "lint.json")
"Ruff exit code: $LASTEXITCODE"
.\.venv\Scripts\python.exe -m pip show coverage radon ruff | Set-Content -Encoding utf8 (Join-Path
$part6Out "tool-versions.txt")
```

Radon output (audit_evidence/midterm_refactor/before/complexity.json) contains no file parse errors. Aggregating top-level function blocks and class methods, excluding class summary blocks and nested closures, gives **342 blocks**, mean CC **4.6082**: A=249, B=61, C=25, D=3, E=4. Highest functions: `load_logmel_cache` 37, `load_profile` 33, `load_mfcc_cache` 33, and `speaker_average` 32. These scores identify review candidates; they do not establish a defect or justify removing validation.

Ruff output (audit_evidence/midterm_refactor/before/lint.json) has exactly **2 findings**, exit code **1**: F541 at `src/audit/for2sec_audit.py:387` (an f-string without placeholders), and F401 at `src/tuning/baseline_fit.py:31` (an unused `BaselineModel` import). This is a limited-rule audit, not an assertion that every possible lint rule passes. Source inspection found that the fixed string has no interpolations and no repository consumer imports `BaselineModel` through `baseline_fit`. These were the first proposed low-risk cleanup and are now implemented in the commit linked below. Cache-helper restructuring is deferred for team review.

The tool version capture (audit_evidence/midterm_refactor/before/tool-versions.txt) has trailing line whitespace removed and UTF-8/LF formatting for Git; all package/version values are unchanged, and both original and stored hashes are recorded. The original capture remains outside the repo. The three raw JSON files are preserved byte-for-byte. Source-relative summaries are in `summary.json`. No `.coverage` database, audio, model, checkpoint or protected dataset is added. The original temporary output folder name was `voice-spoofing-part6-before-20261031-231133`; the reproducibility procedure below explains creating an external `$part6Out` directory. Its generated paths are local evidence locations, not data inputs.

The instrumented 73.21 s test duration and uninstrumented 89.85 s duration are not comparable optimization benchmarks: instrumentation and run conditions differ, and no source refactor occurred.

Implemented cleanup and measured comparison

The focused cleanup is committed as de94333 on `chore/stage11-lint-cleanup-khalil`, stacked on the audit documentation at `3a33105`. The link becomes accessible on GitHub after publication; no PR number is claimed yet. The commit changes two source lines plus its change record. The measured source edits were subsequently committed without additional source changes. Final report/evidence updates are separate documentation changes.

The constant heading lost only its unnecessary `f` prefix; its text is unchanged. The other edit removes the unused `BaselineModel` import alias from a module that still imports the same dependency for other names. Static AST comparison confirmed those are the only source edits. Existing tests were sufficient for these two syntax/import cleanups; no behavior-sensitive function was restructured and no new test was added merely to mirror them.

Full suite, REQUIRE_TORCH_TESTS=1

Before: 937 passed in 89.85s (0:01:29)

After: 937 passed in 57.15s

Interpretation: Same passing count; no skips shown. Durations do not establish a speedup.

Governance test

Before: 3 passed in 0.18s (latest before check; initial 0.22s)

After: 3 passed in 0.18s

Interpretation: No allowlist change.

Coverage-instrumented suite

Before: 937 passed in 73.21s (0:01:13)

After: 937 passed in 85.53s (0:01:25)

Interpretation: Same scope/tools; timings are not optimization benchmarks.

Statement coverage

Before: 3748/4215 = 88.92%

After: 3748/4215 = 88.92%

Interpretation: Unchanged.

Branch coverage

Before: 1047/1312 = 79.80%

After: 1047/1312 = 79.80%

Interpretation: Unchanged.

Combined coverage

Before: 86.76%

After: 86.76%

Interpretation: Unchanged aggregate.

Radon complexity

Before: 342 blocks, mean 4.6082, maximum 37

After: 342 blocks, mean 4.6082, maximum 37

Interpretation: Every measured block's complexity unchanged.

Scoped Ruff

Before: 2 findings, exit 1

After: 0 findings, exit 0; JSON [ ]

Interpretation: F541 and F401 removed. No formatter assessment.

API latency/resource use

Before: Not measured before

After: Not measured after

Interpretation: No performance improvement claimed.

Exact commands from Khalil's after checks (environment variable setup and captures preserved in the report/evidence):

```
$env:REQUIRE_TORCH_TESTS = "1"
.\.venv\Scripts\python.exe -m pytest -q -ra
.\.venv\Scripts\python.exe -m pytest -q tests/governance/test_deepvoice_access.py

937 passed in 57.15s
3 passed in 0.18s

$part6After = Join-Path $env:TEMP ("voice-spoofing-part6-after-" + (Get-Date -Format "yyyyMMdd-
HHmmss"))
New-Item -ItemType Directory -Path $part6After | Out-Null
$env:COVERAGE_FILE = Join-Path $part6After ".coverage"
$env:REQUIRE_TORCH_TESTS = "1"
.\.venv\Scripts\python.exe -m coverage run --branch --source=api,src -m pytest -q -ra
.\.venv\Scripts\python.exe -m coverage report --precision=2
.\.venv\Scripts\python.exe -m coverage json -o (Join-Path $part6After "coverage.json")

937 passed in 85.53s (0:01:25)
TOTAL  4215  467  1312  191  86.76%

$env:RADONFILESENCODING = "UTF-8"
.\.venv\Scripts\python.exe -m radon cc -j api src | Set-Content -Encoding utf8 (Join-Path $part6After
"complexity.json")
.\.venv\Scripts\python.exe -m ruff check --isolated --no-cache --target-version py313 --select
E4,E7,E9,F --output-format json api src | Set-Content -Encoding utf8 (Join-Path $part6After
"lint.json")
"Ruff exit code: $LASTEXITCODE"
.\.venv\Scripts\python.exe -m pip show coverage radon ruff | Set-Content -Encoding utf8 (Join-Path
$part6After "tool-versions.txt")

Ruff exit code: 0
```

Radon emits JSON into its captured file, rather than a numeric console summary. The after summary (audit_evidence/midterm_refactor/after/summary.json) records calculated values, source hashes, raw-file hashes and provenance. After evidence notes (audit_evidence/midterm_refactor/after/README.md) distinguish byte-preserved JSON from normalized console/version text. Captured console (audit_evidence/midterm_refactor/after/console.txt) preserves the full coverage table and static-tool commands; the plain full-suite/governance results were supplied as screenshots. Tool versions match the before capture exactly. The Python version was measured before, not reprinted in the after console; after checks used the same local environment.

Existing API/UI and ML measurements collected from GitHub

The following values are historical repository evidence inspected at the pinned main SHA. They were not rerun for this audit and do not establish an improvement caused by Khalil's proposed refactor. Model-quality figures are validation results only. No additional teammate measurement is needed to reproduce these documented values in this report.

API correctness

Recorded evidence: 50 API tests passed on Python 3.13.2. Iheb checked `/api/status` = HTTP 200 and `/api/nope` = HTTP 404 with the approved error body.

Conditions and provenance: API skeleton; status response uses fixed mock values. Recorded launch command: `uvicorn api.main:app --port 8001`. The exact API-test invocation is not given in the PR.

Source / commit or PR: PR #66, merged as `d9e8c4a`

Limitation: Response time and API memory usage: not measured before. These checks do not demonstrate working inference or a complete MVP.

UI verification

Recorded evidence: 49/49 automated browser checks passed, no console errors; axe-core 4.10 reported 0 violations on 12 screens.

Conditions and provenance: Mock-data prototype v4, Playwright/Chromium, Africa/Tunis, 2026-10-29; widths 1440, 1024, 768, 640 and 390 px. Exact test script/invocation is not included in the handoff.

Source / commit or PR: HANDOFF §13 (design/HANDOFF.md), PR #64

Limitation: Design prototype evidence; manual keyboard, real screen-reader and new-user interpretation reviews remain unverified. No production React performance measurement.

Existing UI correction, credited to Iheb's work

Recorded evidence: Dark-mode hovered-primary-button contrast changed from 4.49:1 to 7.0:1.

Conditions and provenance: Same UI element and state, as recorded in the v4 handoff/PR.

Source / commit or PR: HANDOFF §10 (design/HANDOFF.md), PR #64

Limitation: Previously completed UI correction; not Khalil's refactor and not an execution-speed improvement.

Preprocessing throughput

Recorded evidence: 256 train clips processed in 2.8235499429997617 s; 90.66600739069045 clips/s.

Conditions and provenance: Stage 6, Python 3.13.15, Linux, `preprocessing-v1`, clean code commit `6e1718547756ef04744ce0b3b1b73bb839920052`.

Source / commit or PR: benchmark.json (audit_evidence/stage6_preprocessing/benchmark.json), PR #34

Limitation: Historical train-subset benchmark. Exact invocation is not recorded in this JSON; not rerun. No comparable after measurement exists.

Stage 6 compute dry run

Recorded evidence: Batch 256: mean step 0.05665362309996454 s, peak allocated GPU memory 901,030,912 B; total VRAM 15,637,086,208 B.

Conditions and provenance: Tesla T4, torch 2.11.0+cu130; `dry_run_no_optimizer`, zero optimizer steps, no checkpoint.

Source / commit or PR: Same Stage 6 benchmark and commit

Limitation: Benchmark network lacks the BatchNorm later added to the CNN (project log Entry 79). Do not compare its memory against training as a refactor gain.

Actual CNN training resources

Recorded evidence: Selected four-block, lr 0.001, seed-0 run: peak allocated GPU memory 1,894,575,616 B; median epoch about 5.0 s; 15 epochs totaling about 75 s.

Conditions and provenance: Tesla T4, Python 3.13.16, torch 2.11.0+cu130, CUDA 13.0, batch 256; code `9be2f2545520432db08a0d083a06a668cd84103a`. Timing is rounded in the results document; exact per-epoch values are in the selection report.

Source / commit or PR: CNN results §6 (results/cnn_selection.md), run_report.json (audit_evidence/stage8_cnn/run_report.json), selection_report.json (audit_evidence/stage8_cnn/selection_report.json)

Limitation: Training resources, not API inference latency. No same-conditions refactor comparison exists. Original run used notebook 04; this audit did not execute it.

Baseline validation quality

Recorded evidence: Selected C=10, gamma=0.1: val-A EER 0.0014925373134327957 (about 0.1493%); val-B EER 0.0 on 1,329 clips.

Conditions and provenance: 12,404 train clips, 1,330 val-A clips; config selected on val-A; D29 threshold fitted on val-B. Code `297c352c91bfc9daafa2943ba04154194aa33248`.

Source / commit or PR: Baseline results (results/baseline.md), selection report (audit_evidence/stage7_baseline/selection_report.json), threshold report (audit_evidence/stage7_baseline/threshold_report.json)

Limitation: Existing validation results; no FoR test or external generalization claim. Not a refactor outcome.

CNN validation quality

Recorded evidence: All three selected-configuration seeds: best val-A EER 0.0 and ROC-AUC 1.0; bundle seed 0.

Conditions and provenance: Configuration selected on val-A; Stage 8 code `9be2f25`.

Source / commit or PR: CNN results §4 (results/cnn_selection.md), Stage 8 selection report

Limitation: Selection-set results can be optimistic; do not claim superiority or a refactor benefit.

Risk-policy validation

Recorded evidence: Achieved deferral 0.09932279909706546 = 132/1,329 val-B clips; threshold 0.6333105553100875, half-width 0.3636377690579665.

Conditions and provenance: `risk-v1`, bundle seed 0, CPU scoring, code `11c07fedcd755dc72f16b2695f180cf094aeca04`; policy fitted on val-B.

Source / commit or PR: Risk results (results/risk_policy_fit.md), run_report.json (audit_evidence/stage10_risk/run_report.json)

Limitation: Existing fitted values are recorded for context and remain untouched. No new fit, score, or protected-data access occurred.

The source JSON values were cross-checked against their corresponding results documents. A recorded command absent from those sources is explicitly marked unavailable; no invocation or measurement is reconstructed as if it had been executed in this audit.

Source-arithmetic checks performed in PowerShell from the repository root (no model or benchmark execution):

```
$benchmark = Get-Content docs/audit_evidence/stage6_preprocessing/benchmark.json -Raw | ConvertFrom-
Json
$selection = Get-Content docs/audit_evidence/stage8_cnn/selection_report.json -Raw | ConvertFrom-Json
$epochTimes = @($selection.seeds.'0'.history.epoch_seconds | Sort-Object)
$benchmark.preprocessing.rows / $benchmark.preprocessing.seconds
($epochTimes | Measure-Object -Sum).Sum
$epochTimes[[int][Math]::Floor($epochTimes.Count / 2)]
```

Numeric results (represented with a decimal point):

```
90.66600739069045
75.26422497099998
5.019009443000186
```

The median expression above is for this recorded run's odd count of 15 epochs. It is not a general even-count median implementation. These values were also independently read from the source JSON and checked against the rounded results document.

These calculations explain the existing results document's rounded 75 s training time and 5.0 s median epoch time. They are not new timing measurements.

Initial code audit and proposed refactoring list

The read-only audit reviewed the feature-cache pair, preprocessing validation, Stage 17 API skeleton, per-clip score storage, and the Stage 10 risk runner at main SHA `98fd8d7e17f81ff1f9687f6ea3a83804a50fa628`. It identified the feature-cache modules' deliberate private-helper reuse as a maintainability candidate, not a demonstrated defect. Structural change in the other behavior-sensitive areas is not justified by measured maintenance or performance evidence.

src/audit/for2sec_audit.py + src/tuning/baseline_fit.py

Original problem / observation: Ruff identifies one unnecessary f-string prefix and one unused import.

Evidence before: F541 at audit line 387 and F401 at baseline-fit line 31; 2 total findings under E4/E7/E9/F.

Refactoring performed / proposed: Removed only the unused `f` prefix and unused `BaselineModel` import on `chore/stage11-lint-cleanup-khalil`; literal text and fitting logic preserved.

Evidence after: Scoped Ruff findings 2 to 0; coverage and complexity unchanged.

Test result: After: 937 passed; governance 3 passed; coverage run 937 passed.

Commit/PR: de94333; PR pending.

Remaining limitation: These are small code-quality improvements; no complexity or performance gain is assumed.

src/features/mfcc_cache.py + src/features/logmel_cache.py

Original problem / observation: The log-Mel module imports private constants and validators from the MFCC-specific module, coupling two feature caches by implementation detail.

Evidence before: `logmel_cache.py` imports `_check_split`, `_check_hex`, `_split_rows`, `_meta_text`, and schema constants from `mfcc_cache.py`. The matching test files define 20 MFCC test functions, 22 log-Mel test functions, and one code-commit-independent NPZ-byte test function.

Refactoring performed / proposed: Proposed for team review: move only the genuinely shared cache constants/helpers into a neutral internal module, then have both caches import it. Preserve validation order, exception text, JSON/NPZ bytes, and feature-specific schema. No speedup is claimed.

Evidence after: N/A; no code changed.

Test result: If approved: rerun the three cache test files, full suite, and governance test; verify byte-for-byte cache outputs remain identical. The complete baseline passed; checks after the proposed refactor are pending.

Commit/PR: None; report branch exists, no refactor commit or PR.

Remaining limitation: The cache formats and evidence are governed; helper extraction can still change observable errors or bytes and needs review before implementation.

src/preprocessing/validation.py

Original problem / observation: The `run()` orchestrator coordinates manifest processing, shortcut and padding checks, invariance checks, and report assembly in one function.

Evidence before: `run()` is 79 lines (312-390) in a 410-line module. It already calls focused helpers; measured CC is 6 (rank B), and no maintenance failure was found.

Refactoring performed / proposed: No refactor recommended now. Keep the audit steps visible together unless reviewers identify a specific maintenance problem and agree on exact-output regression checks.

Evidence after: N/A; no code changed.

Test result: Existing `tests/preprocessing/test_validation.py`; no code change; included in the complete baseline.

Commit/PR: None.

Remaining limitation: It validates the frozen preprocessing profile; changing its order or report can alter governed evidence.

api/config.py

Original problem / observation: Three input parameters lack annotations: `path` in `load_api_config` and both parameters of `_type_ok`.

Evidence before: `load_api_config(path=None) -> ApiConfig` and `_type_ok(value, expected) -> bool`; 88-line module. No static type checker/configuration is present in the current-main tree.

Refactoring performed / proposed: Lower-priority candidate: add accurate input annotations only; keep behavior and error text unchanged. No split is recommended because the loader's validation steps are cohesive.

Evidence after: N/A; no code changed.

Test result: Existing `tests/api/test_api_config.py` is the behavior guard; no code change; included in the complete baseline.

Commit/PR: None; report branch exists, no refactor commit or PR.

Remaining limitation: Small maintainability gain only; no performance gain or type-check result can be claimed without a configured checker.

api/errors.py

Original problem / observation: Error mapping is spread across four handlers, but each covers a distinct FastAPI/Starlette condition and already uses shared error_body.

Evidence before: 75-line file; shared response-shape construction already exists.

Refactoring performed / proposed: No refactor recommended in this pass; combining handlers could obscure distinct header, status, and logging behavior.

Evidence after: Not applicable.

Test result: Existing API error and middleware tests remain the evidence if later changed.

Commit/PR: None.

Remaining limitation: Retain current separation unless a concrete duplication issue is identified.

src/evaluation/clip_scores.py

Original problem / observation: Serialization, validation, integrity checks, and file creation sit in one module.

Evidence before: 391-line governed implementation; writer/reader already share validation and canonical-byte helpers. Exact hashes, bytes, refusal rules, and exclusive file creation are tested.

Refactoring performed / proposed: No refactor recommended in this pass. Do not split this code without a specific measured maintenance problem and byte-for-byte regression tests.

Evidence after: Not applicable.

Test result: tests/evaluation/test_clip_scores.py covers canonical output and refusal cases.

Commit/PR: None.

Remaining limitation: High risk because output bytes and stored evidence are frozen.

src/tuning/risk_run.py

Original problem / observation: The run orchestrator performs provenance checks, cache loading, scoring, fitting, diagnostics, and output writing.

Evidence before: 298-line Stage 10 runner; execution order and committed evidence are governed. Repeated score and policy-fit calls are intentional determinism checks, not removable duplicate work.

Refactoring performed / proposed: No refactor recommended in this pass. Any later extraction must preserve exact call order and output bytes.

Evidence after: Not applicable.

Test result: tests/tuning/test_risk_run.py and committed evidence would need to remain unchanged.

Commit/PR: None.

Remaining limitation: No timing baseline exists; do not claim faster execution or rewrite evidence.

Initial recommendation and team review gate

This audit does not establish a measurable performance problem. The two measured Ruff findings have been cleaned up and tested. The private-helper coupling between the two feature-cache modules is a larger candidate for later team review; it is a structural cleanup, not a performance optimization. The lower-priority `api/config.py` typing gap is optional. Only the two lint edits have been made. The cache refactor should wait for team review because it touches governed schemas and deterministic evidence. Storage or run-orchestration changes are out of scope without explicit team agreement.

Evidence and review status

• Iheb's existing API/UI checks and UI correction are documented in PRs #64 and #66 and have been included above.

• Donia's existing ML quality, timing and resource evidence is committed under `docs/results/` and `docs/audit_evidence/` and has been included above.

• The new software baseline now includes coverage, complexity and lint evidence. The two-edit before/after comparison is recorded below. API response-time/resource measurements remain unmeasured before and after; the existing evidence does not justify estimating them.

• No team approval or implementation PR for the newly proposed cache-helper extraction has been verified in this audit. The initial audit must reach the team before this evidence-sensitive change is implemented, as requested in the task brief.

• Khalil's human review and Malak's final editorial review remain to be recorded.

Final submission requirements and measurement plan

The supplied Midterm Instructions, section E, require refactors applied through PRs and human review with before/after metrics. The mandatory-deliverables section also requests repository content plus PDF. The implementation guidance names a coverage target of at least 60%, subject to course policy. The measured statement, branch and combined coverage each exceed 60% for the explicitly stated `api/` and `src/` scope. This establishes the recorded baseline coverage result for that scope; it does not establish per-file coverage attainment or instructor acceptance of the scope. This report documents a completed small cleanup with actual before/after checks. Its scope is modest: two lint fixes, with no measured performance improvement. Whether that scope satisfies the assignment must be confirmed by the team/instructor; it is not represented as a major structural refactor.

The companion `refactor-optimization-report.pdf` is regenerated from this Markdown, including the after-results and review limitations. Team review, required CI and final acceptance must be completed before treating it as an approved submission.

Coverage, Radon and Ruff are measurement tools only; they are not new application dependencies. The measured baseline above was collected through the equivalent individual commands supplied to Khalil. The following procedure documents creating an external output folder and collecting the same categories; it is a reusable procedure, not a claim that the entire block was executed verbatim. Run from the repository root before any code refactor, preserve the outputs outside the repository, and keep the captured tool versions for the after-run. Copy commands from the Markdown version; PDF code lines can wrap for display. If the run fails, retain its output and resolve the cause before interpreting coverage.

```
.\.venv\Scripts\python.exe -m pip install coverage==7.17.0 radon==6.0.1 ruff==0.17.3
if ($LASTEXITCODE -ne 0) { throw 'Audit-tool installation failed' }
$part6Out = Join-Path $env:TEMP ('voice-spoofing-part6-before-' + (Get-Date -Format 'yyyyMMdd-HHmmss'))
New-Item -ItemType Directory -Path $part6Out | Out-Null
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m pip show coverage radon ruff | Set-Content -Encoding utf8 (Join-Path
$part6Out 'tool-versions.txt')
.\.venv\Scripts\python.exe -m pip check
if ($LASTEXITCODE -ne 0) { throw 'Dependency check failed' }
git rev-parse HEAD | Set-Content (Join-Path $part6Out 'commit.txt')
git status --short | Set-Content (Join-Path $part6Out 'working-tree.txt')
$env:REQUIRE_TORCH_TESTS = '1'
$env:COVERAGE_FILE = Join-Path $part6Out '.coverage'
.\.venv\Scripts\python.exe -m coverage run --branch --source=api,src -m pytest -q -ra
if ($LASTEXITCODE -ne 0) { throw 'Tests failed; do not report this as a passing baseline' }
.\.venv\Scripts\python.exe -m coverage report -m
.\.venv\Scripts\python.exe -m coverage json -o (Join-Path $part6Out 'coverage.json')
$env:RADONFILESENCODING = 'UTF-8'
.\.venv\Scripts\python.exe -m radon cc -s -a api src
.\.venv\Scripts\python.exe -m radon cc -j api src | Set-Content -Encoding utf8 (Join-Path $part6Out
'complexity.json')
.\.venv\Scripts\python.exe -m ruff check --isolated --no-cache --target-version py313 --select
E4,E7,E9,F --output-format json api src | Set-Content -Encoding utf8 (Join-Path $part6Out 'lint.json')
$part6RuffExit = $LASTEXITCODE
if ($part6RuffExit -gt 1) { throw 'Ruff failed to execute correctly' }
Write-Output ('Measurement output: ' + $part6Out)
```

Capture the test summary and displayed coverage/complexity output as well as the generated files. Ruff exit 1 means findings were reported; no `--fix` is used. The chosen limited Ruff rule set is an audit scope, not a replacement coding policy. Report the scope as `api/` and `src/`, with no test/notebook files included in the coverage denominator. From `coverage.json`, record statement coverage (`covered_lines / num_statements`) separately from branch coverage and the tool's combined percentage; assess the course's 60% target against the agreed coverage definition. Coverage.py does not automatically prove that every subprocess was traced. Record subprocess and unexecuted-path limitations rather than hiding them. The instrumented run's duration is not comparable to the uninstrumented 89.85 s baseline as a speed measurement.

After an agreed code refactor, run the same procedure with an `after` output directory, the same installed tool versions, runtime and scope. Link that refactor's actual commit or PR. Count only lint findings actually removed, compare complexity for the same functions, and report unchanged metrics honestly. A neutral helper move may improve dependency direction without changing complexity, coverage or execution speed. No performance claim should be added without an independently justified benchmark and the same conditions before and after.

Tool references: Coverage.py execution, JSON reporting, Radon CLI, Ruff configuration.

Completion gates still open: publish the audit for the team; open the focused cleanup PR, obtain required CI and human code/report review, and record final acceptance. The cleanup is implemented and measured, and the PDF is checked against this Markdown. No high-risk structural refactor was attempted while team review is pending. These checks cannot establish 100% correctness.

Closing record

• Improvements achieved: two measured lint findings removed in the linked cleanup commit; no change to aggregate coverage or any measured complexity block. Repository API/UI and ML evidence is traced to its sources. No performance improvement is claimed; the historical UI contrast correction belongs to Iheb.

• Intentionally unchanged: model behavior, preprocessing rules, thresholds, API contracts, frozen evidence, governance decisions, datasets, audio and test splits.

• Remaining technical debt: four rank-E functions, cache-helper coupling, optional API input annotations and individual coverage gaps. API latency and resource use remain unmeasured. The larger cache refactor is deferred for team review.

• Measurement limitations: full before and after runs each show 937 passing tests with no skips. Khalil executed project-runtime checks because the assistant could not launch the project interpreter. Timings are observations rather than speed benchmarks; lint covers only E4/E7/E9/F, and coverage excludes tests/notebooks. Passing tests and a coverage target do not prove correctness or instructor acceptance.

• AI tools used: Codex for inspection, the two lint edits, evidence comparison, report drafting and PDF generation. No application feature or model optimization was generated.

• Human review: Khalil ran the full suite, governance checks and measurement commands, supplied their outputs, and committed the cleanup. These actions validate the recorded execution; they do not establish independent code/report acceptance. Khalil must understand the two-line diff and report; teammate code review, Malak's editorial review, required CI and final acceptance remain pending. Publication/PR numbers will be recorded after the branches are pushed.

Source Markdown SHA-256: 4266ec73f063990d53b43e6e15b973d23434aea2e50b670f9d0878968c37cb1d